

Constructor University Bremen
Fall Semester 2025

Robotics and Intelligent Systems Lab
Michael Görner

Assignment 02 – RViz and Closed Loop Motion Controllers

Ali Ahmed Mohamed Amaal Sayed Labib Ibrahim
Yassein Ahmed Mahmoud
Yassin Baher Youssef

Date of Execution: September 19th, 2025

Initial Exploration



Figure 1: RViz interface screenshot; TopDownOrtho, LaserScan Style: Points, LaserScan Decay Time: 0, LaserScan Color Transformer: AxisColor, LaserScan Size(Pixels): 15

1. /scan Properties:

- **Style (Points/Flat Squares)**
 - Effect: Returns from the 2D Lidar render as filled squares instead of sub-pixel points, improving spatial continuity of thin walls in the display.
 - When useful: In sparse scans or when debugging wall outlines and gaps (doorways), the squares make discontinuities more obvious.
- **Size (Pixels)**
 - Effect: Increasing the size makes the display appear as larger dots, which subjectively thickens walls and makes occupancy discontinuities easier to see; decreasing produces a crisp but potentially “speckled” map edge.
 - When useful: Larger size is helpful to quickly notice obstacles while teleoperating; smaller size is better for precise inspection of thin structures and

debugging.

- **Decay Time**
 - Effect: In RViz's LaserScan display, Decay Time is the number of seconds each point remains visible: RViz retains hits from the last N seconds so recent scans accumulate into a short-lived trail that fades as points age.
 - When useful: Great for detecting intermittent sensor dropouts and for visualizing motion patterns without relying on a SLAM map.
- **Color Transformer (Intensity / Range / AxisColor)**
 - Effect:
 - Color Transformer controls how each point is colored: Intensity shades by the sensor's return strength (bright = strong reflector, dark = weak).
 - Range maps color to distance (so nearby obstacles stand out instantly).
 - AxisColor colors point by their orientation in the global axes (R = X, G = Y, B = Z), which is most useful for 3D spatial context.
 - When useful: Range coloring is especially helpful for immediately spotting the closest barriers within the field of view.

Move and Tell

1. *turtle_teleop_key* outputs to */turtle1/cmd_vel*, while the robot in the Flatland setup listens on */cmd_vel* without the *turtle* namespace. The topic remap above is therefore necessary otherwise the simulated robot receives no commands.

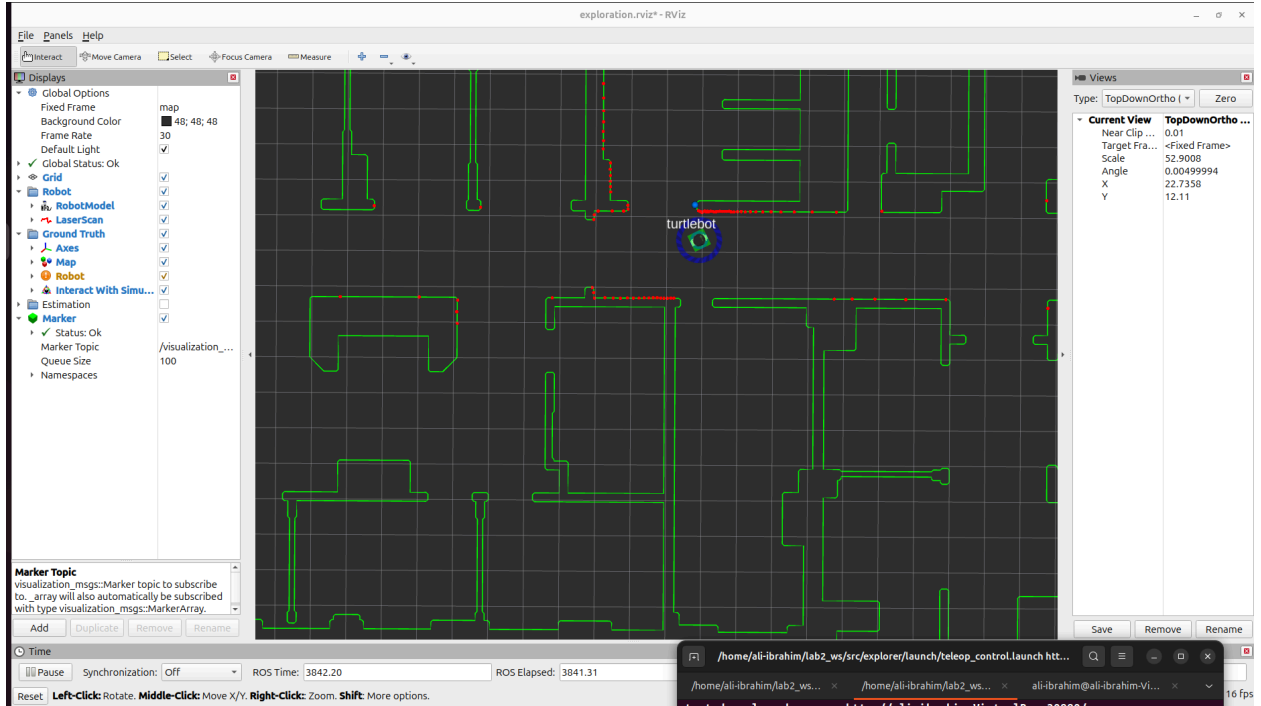


Figure 2: Blue sphere 1m away from simulated robot

Autonomous Exploration

1. Algorithm:

The controller implements a reactive exploration behavior by analyzing the front-facing laser scan data at each time step. It identifies the nearest obstacle as a potential collision threat and the furthest detected point as the most promising direction for exploration. When obstacles are detected closer than the minimum safe distance threshold, the controller enters a defensive mode where it rotates away from the threat while reducing forward speed. Otherwise, it maintains forward motion while steering proportionally toward the direction of maximum detected range.

2. Controller I/O:

- **Subscribes:** `/scan` (sensor_msgs/LaserScan)
- **Publishes:** `/cmd_vel` (geometry_msgs/Twist)
- **Publishes (debug):** `/visualization_markers`
- **Parameters (private):**
 - `forward_speed` (`v_max`, default 0.30 m/s)
 - `rotation_speed` (`ω_max`, default 0.80 rad/s)

- min_safe_distance (d_{safe} , default 0.50 m)
- max_scan_angle (θ_{max} , default 1.57 rad)

3. Marker Messages:

- **Front FOV rays:** orange lines showing the considered scan window.
- **Nearest obstacle:** a small red sphere at (r_{min}, α_{min}) in base_link.
- **Most open direction:** a green arrow from the robot pointing toward α_{max} with length proportional to r_{max} .
- **ALTERNATIVE:** have the “turtlebot” text marker act as a proximity status indicator, changing colors based on whether the simulated robot is in d_{safe} or not.

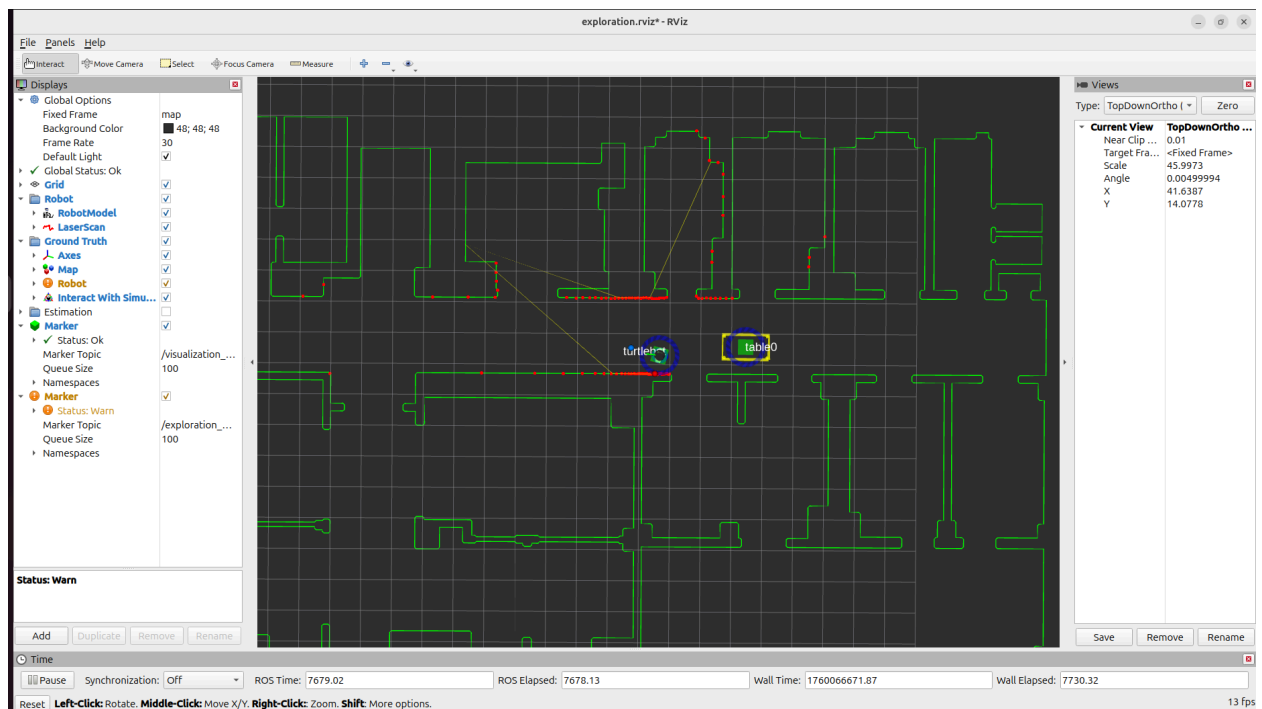


Figure 3: LaserScan range (“scan window”) visualized through orange lines

4. Challenges and Solutions

1.

- **Challenge:** During testing, the robot frequently exhibited oscillatory behavior when attempting to pass through narrow doorways. In Figure 4, the robot can be seen positioned directly in front of a doorway opening. The marker visualization reveals the cause of this behavior: the two doorframe sides appear as nearly equally close obstacles, marked by the red sphere rapidly switching between the left and right sides across consecutive scans. This causes the robot's target direction arrow to swing back and forth, resulting in a rotation command that alternates between left and right turning. The robot rocks back and forth in front of the doorway without successfully passing through. This problem arises from the purely reactive nature of the controller, which responds immediately to each new scan without any memory or temporal smoothing of control outputs.
- **Solution:** Implementing temporal smoothing of control commands, where the published velocity represents a weighted average of the current calculated velocity and recent historical velocities. This would dampen rapid oscillations while still allowing the robot to respond to actual changes in the environment.

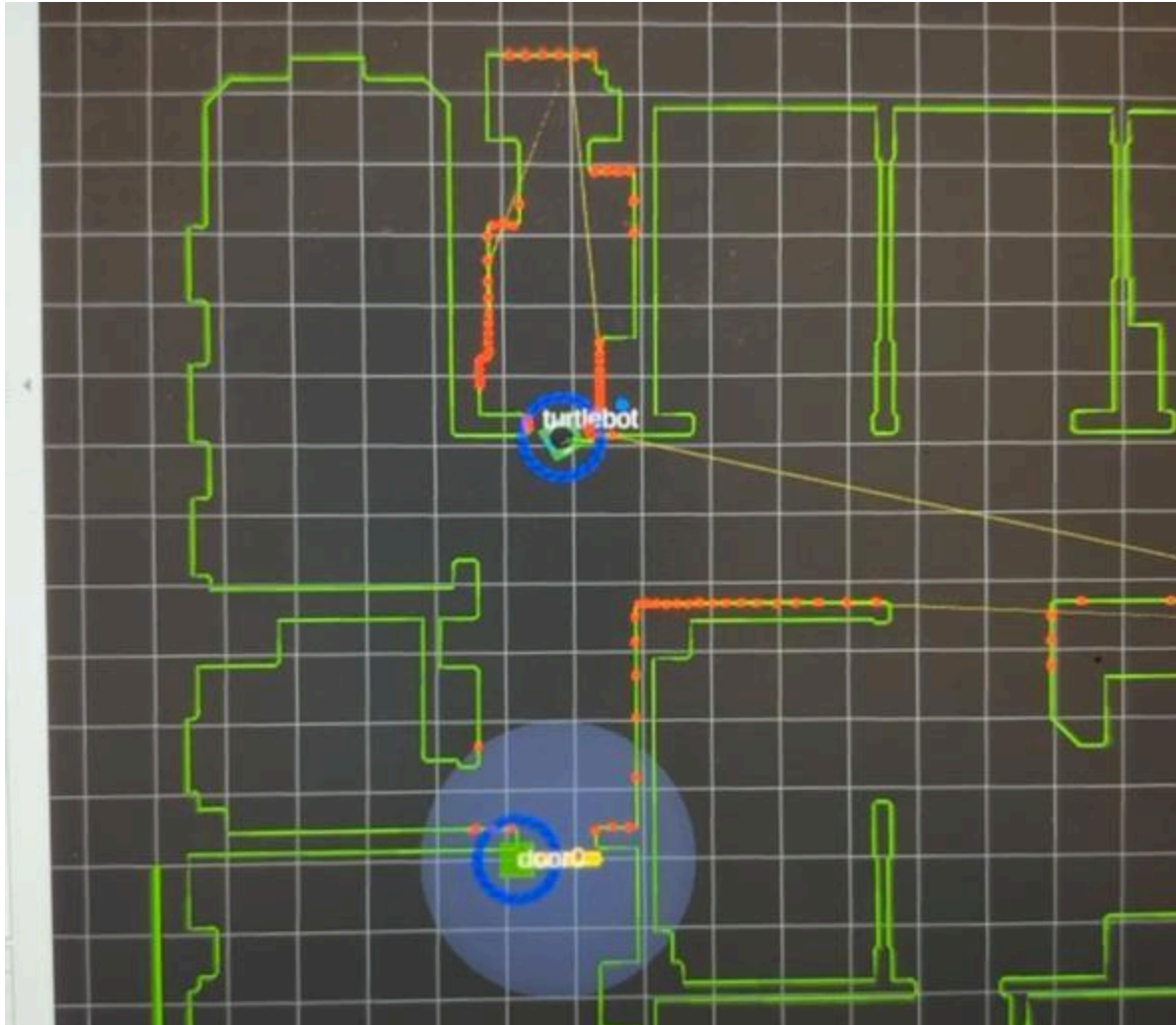


Figure 4: turtlebot stuck in doorway

- **Challenge:** “Go to largest gap” is too short-sighted: it keeps picking the widest open space right now, so the robot drifts back into big rooms and avoids narrow doors that lead to the last 20% of the map. This causes loops in halls/cul-de-sacs and a hard ~80% coverage ceiling after hours.
- **Solution:** A 360° scan can break local loops.

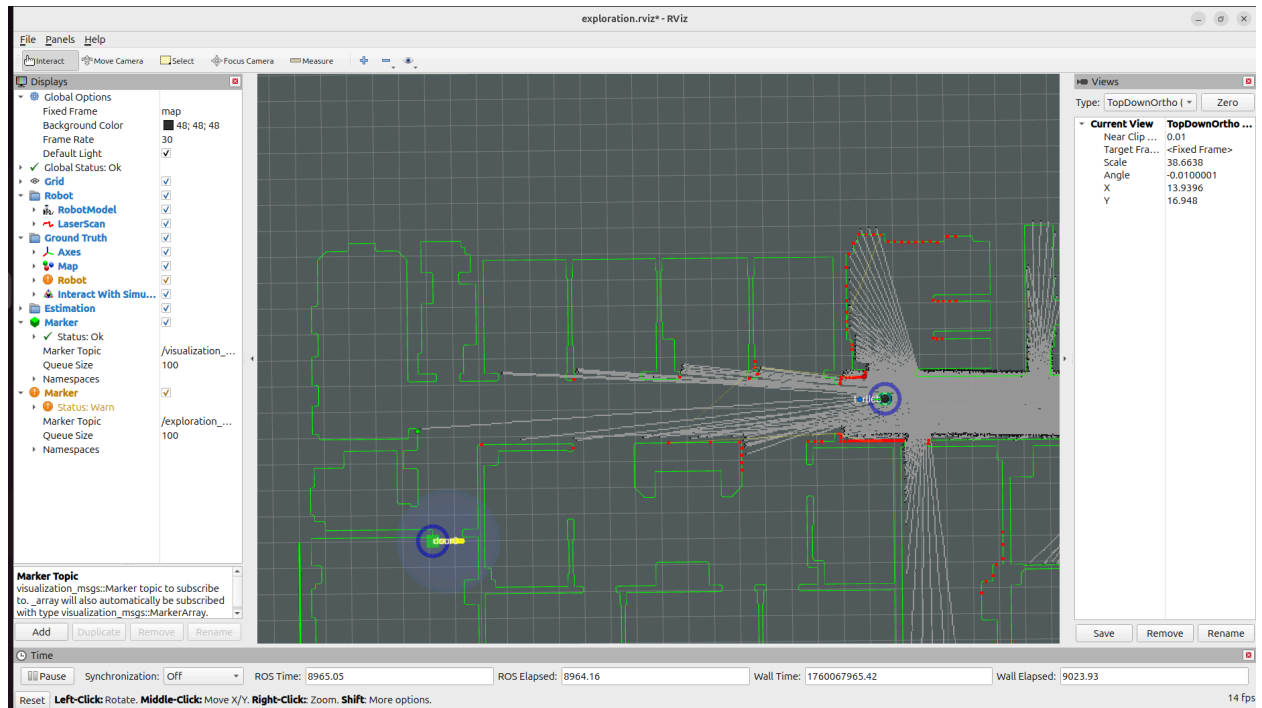


Figure 5: RViz interface screenshot showing how little ground turtlebot covered

Contributions:

- **Initial Exploration - Yasin ElBanna**
 - Wrote, ran, and initialized nodes.
 - Lab report section
- **Move and Tell - Yassin Baher**

- Wrote, ran, and initialized nodes.
 - Lab report section
- **Autonomous Exploration - Ali Ibrahim**
 - Wrote parameters file, wrote, ran, and initialized nodes.
 - Lab report section

External Sources

- https://www.cs.cmu.edu/~motionplanning/papers/sbp_papers/integrated1/yamauchi_frontiers.pdf
- https://wiki.ros.org/explore_lite

